PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No. 8 (A)

Aim : To implement any parsing technique.

Objective: Develop a program to implement

a. Predictive parser
b. Operator precedence parser

Outcome: Students are able to understand various parsing techniques. Also they are able to implement a program to generate parsing table for respective technique.

Theory:

A special case of top-down parsing without backtracking is called a predictive parsing. While writing grammar if we eliminate left recursion from it, and left factoring the grammar, we can obtain a grammar that can be parsed by a recursive-descent parser without backtracking is called a predictive parser.

A non-recursive predictive parser is build using stack. The main problem in predictive parsing is, how to decide which production to be applied for a non-terminal. The non-recursive parser given in figure 8.12, looks for the production to be applied in parsing table which is constructed from certain grammars.

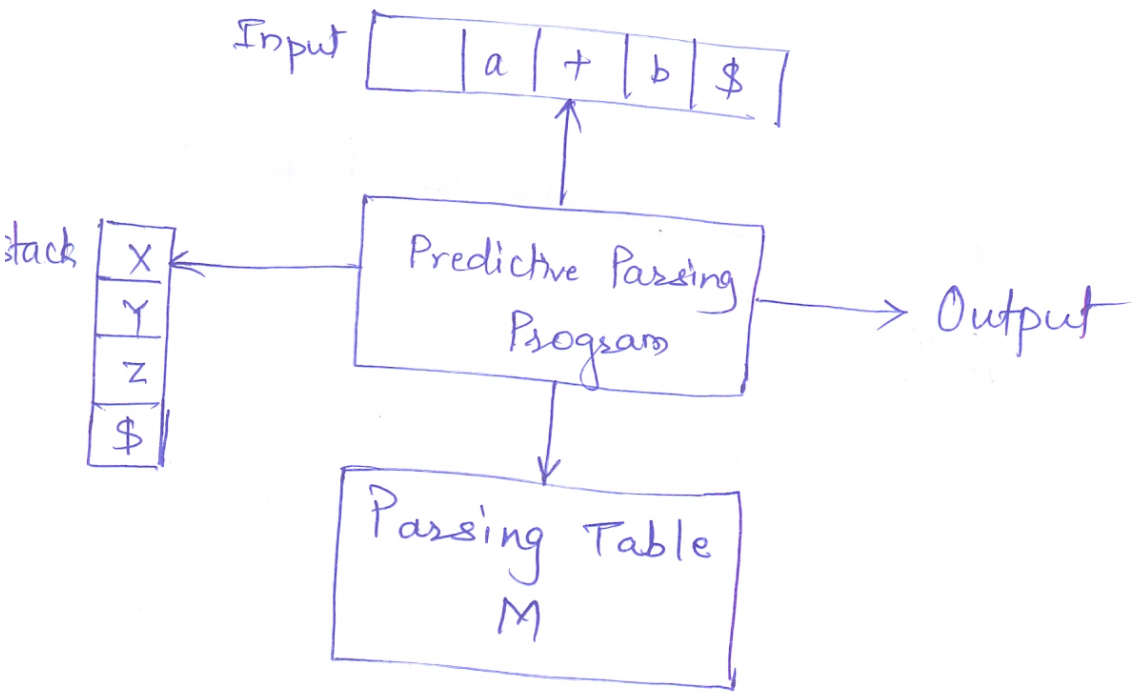


FIGURE: Model of a non-recursive predictive parser

Algorithm- Construction of predictive parsing table

Input- Grammar G

Output- Parsing Table

Method-

1. For each production A□α of the grammar do steps 2 and 3
2. For each terminal a in FIRST(α), add A□α to M[A,a]
3. If ε is in FIRST(α), add A□α M[A,b] for each terminal b in FOLLOW(A).
4. If ε is in FIRST(α) and $ is in FOLLOW(A), add A□α to M[A,$]
5. Make each undefined entry of M error.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No.: **C-54**	Name: **Quazi Md Moinuddin**
Class:**TE-C**	Batch: **C-3**
Date of Experiment:	Date of Submission: **14-04-2025**
Grade:	

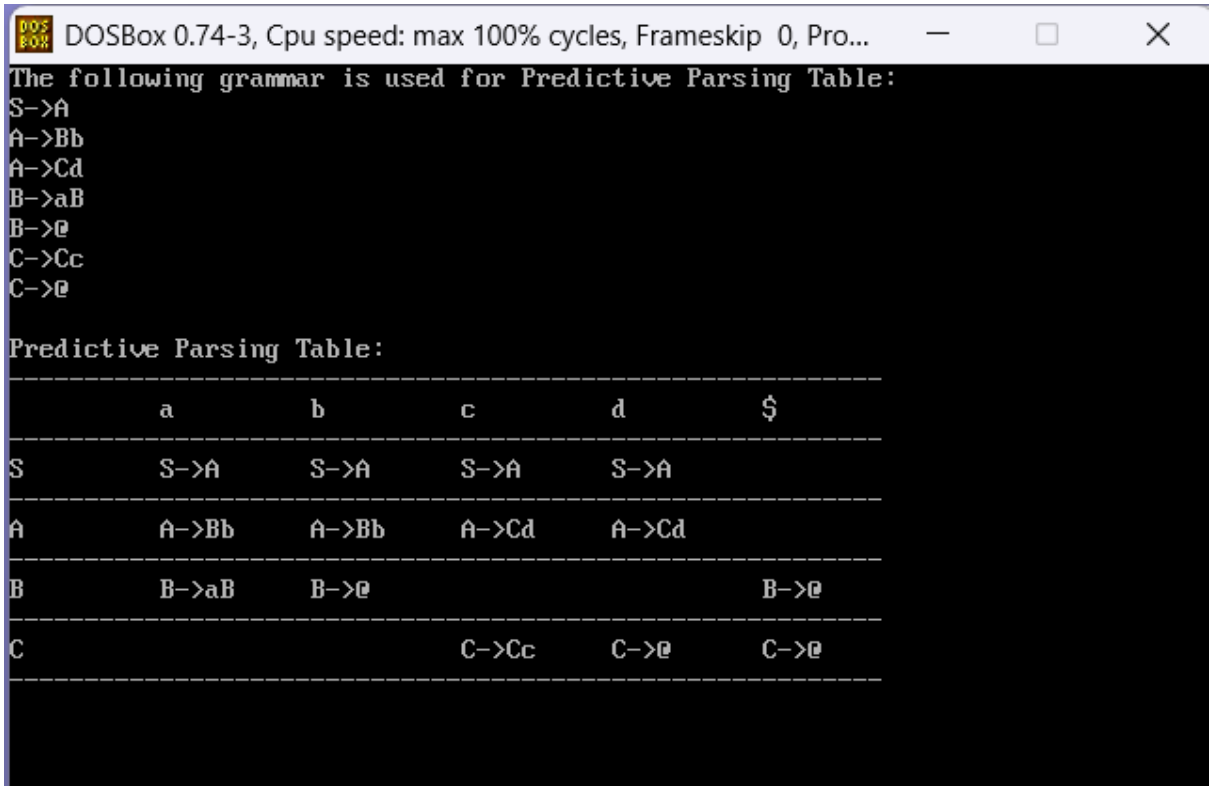
B.1 Software Code written by student:

```
#include <stdio.h>
#include <string.h>
#include <conio.h>

char prol[7][10] = { "S", "A", "A", "B", "B", "C", "C" };
char pror[7][10] = { "A", "Bb", "Cd", "aB", "@", "Cc", "@" };
char prod[7][10] = { "S->A", "A->Bb", "A->Cd", "B->aB", "B->@", "C->Cc", "C->@" };
char first[7][10] = { "abcd", "ab", "cd", "a@", "@", "c@", "@" };
char follow[7][10] = { "$", "$", "$", "a$", "b$", "c$", "d$" };
char table[5][6][10];

int numr(char c) {
   switch (c) {
      case 'S': return 0;
      case 'A': return 1;
      case 'B': return 2;
      case 'C': return 3;
      case 'a': return 0;
      case 'b': return 1;
      case 'c': return 2;
      case 'd': return 3;
      case '$': return 4;
   }
   return 2;
}

void main() {
   int i, j, k;
   clrscr();

   // Initialize table with blanks
   for (i = 0; i < 5; i++)
      for (j = 0; j < 6; j++)
```

```
        strcpy(table[i][j], " ");

printf("The following grammar is used for Predictive Parsing Table:\n");
for (i = 0; i < 7; i++)
   printf("%s\n", prod[i]);

// Construct parsing table using FIRST
for (i = 0; i < 7; i++) {
   k = strlen(first[i]);
   for (j = 0; j < k; j++) {
      if (first[i][j] != '@') {
         int row = numr(prol[i][0]) + 1;
         int col = numr(first[i][j]) + 1;
         strcpy(table[row][col], prod[i]);
      }
   }
}

// For ε (epsilon) productions, use FOLLOW
for (i = 0; i < 7; i++) {
   if (strlen(pror[i]) == 1 && pror[i][0] == '@') {
      k = strlen(follow[i]);
      for (j = 0; j < k; j++) {
         int row = numr(prol[i][0]) + 1;
         int col = numr(follow[i][j]) + 1;
         strcpy(table[row][col], prod[i]);
      }
   }
}

// Setup headers
strcpy(table[0][0], " ");
strcpy(table[0][1], "a");
strcpy(table[0][2], "b");
strcpy(table[0][3], "c");
strcpy(table[0][4], "d");
strcpy(table[0][5], "$");

strcpy(table[1][0], "S");
strcpy(table[2][0], "A");
strcpy(table[3][0], "B");
strcpy(table[4][0], "C");

// Display table
printf("\nPredictive Parsing Table:\n");
printf("------------------------------------------------------------\n");
for (i = 0; i < 5; i++) {
   for (j = 0; j < 6; j++) {
      printf("%-10s", table[i][j]);
   }
```

```
        printf("\n----------------------------------------------------------\n");
    }

    getch();
}
```

B.2 Input and Output:



B.3 Observations and learning:

The predictive parsing table was successfully constructed using the given grammar, along with its FIRST and FOLLOW sets. The table entries correctly map non-terminals and input symbols to the corresponding production rules. Epsilon (@) productions were appropriately handled using FOLLOW sets. The implementation validates the LL(1) grammar and is useful for syntax analysis in a compiler.

B.4 Conclusion:

The implementation of the predictive parsing table demonstrates how LL(1) parsing works using FIRST and FOLLOW sets. It helps in understanding syntax analysis and forms the basis for building efficient parsers in compilers.

B.5 Question of Curiosity

(To be answered by student based on the practical performed and learning/observations)

1) What is the mechanism of Top-Down parser ?

Top-down parsing starts from the start symbol and tries to derive the input string by repeatedly applying production rules. It expands non-terminals by choosing appropriate rules using lookahead symbols and attempts to match the input from left to right.

It builds the parse tree from the root to the leaves.

2) How do you recognize LL(1) grammar ?

A grammar is **LL(1)** if:

- It can be parsed from Left to right, producing a Leftmost derivation with 1 lookahead symbol.
- There are no left recursions.
- For each non-terminal, the FIRST sets of its different productions are disjoint.
- If any production derives epsilon (ε), its FIRST and FOLLOW sets are disjoint.

3) What are the key differences between recursive and non recursive-descent parsers ?

Feature	Recursive Descent Parser	Non-Recursive Descent Parser
Uses recursion	Yes (one function per non-terminal)	No (uses parsing table and stack)
Simplicity	Easy to implement manually	More systematic and table-driven
Backtracking	May require backtracking	Usually avoids backtracking (LL(1))
Speed	Slower if backtracking occurs	Faster with predictive parsing